DEPARTMENT: INFORMATION SCIENCE AND ENGINEERING							
Course Code: 24CB340		Course Title : COMPUTATIONAL STATISTICS					
Total Credits: 04 *L-45: T-00: P-30: SL and TW-45*		*SDGs addressed: 4*		*Course Category: PCC*		*Course Type: Integrated*	
Teaching Learning Process		*Total Contact Hours /semester*	*Assessment Weightage and Marks*				
	Lecture	45		CIE	SEE	Total	
	Tutorial	-	*Weightage, %*	*40*	*60*	*100*	
	Practical	30	*Maximum Marks*	*40*	*60*	*100*	
	Self-learning and Teamwork	45	*Minimum Marks*	*20*	*25*	*45**	
	Total	120					

Note: *For passing the course, the student must score a minimum of 45 Marks (CIE+SEE: 20 + 25 or 21 + 24)

COURSE PRE REQUISITES: Statistics

COURSE OBJECTIVES: To introduce computational methods in statistics with emphasis on the usage of statistical tools and give a solid mathematical foundation of computational statistics.

COURSE OUTCOMES: After completing this course, students should be able to:

CO#	Course Outcomes	Highest Level of Cognitive Domain	Working knowledge
CO1	Demonstrate the ability to apply technical skills to solve statistical problems.	L2	WK2,WK8
CO2	Identify and apply appropriate regression models on datasets.	L2	WK2,WK8
CO3	Perform Dimensionality Reduction using statistical techniques.	L3	WK6,WK8
CO4	Demonstrate statistical experiments using Factor analysis methods and techniques.	L3	WK6,WK8
CO5	Illustrate the real time problems using Clustering approaches.	L3	WK6,WK7

L1 – Remember, L2 – Understand, L3 – Apply, L4 – Analyze, L5 – Evaluate, L6 - Create

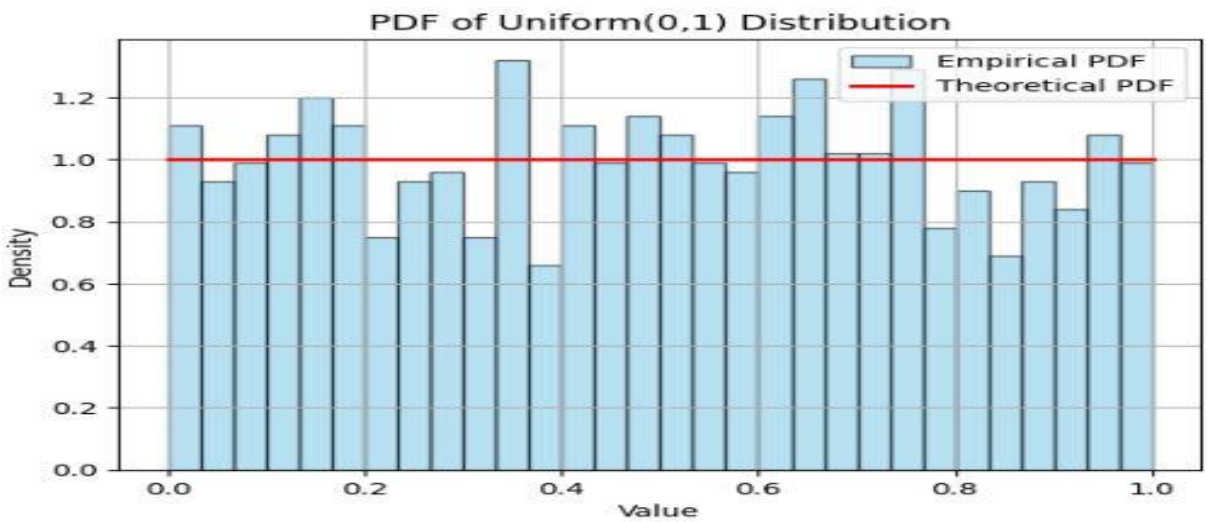
Laboratory

Week	List of Experiments/ Programs	No. of Hours
1	Generate 1000 samples from a uniform continuous distribution between 0 and 1. Plot the PDF (Probability Density Function) using a histogram and overlay the theoretical PDF using python program	2
2	Generate 1000 samples from a normal distribution with a given mean and standard deviation using python program	2
3	Write a Python program to generate 1000 samples from a Normal distribution (mean = 50, std = 10), plot the histogram with the theoretical normal curve, and display the sample mean and standard deviation.	2
4	Write a Python program to generate and plot the Poisson distribution for λ = 4 using scipy.stats.poisson, and visualize the probability mass function (PMF) with a bar chart.	2
5	A dataset containing the prices of houses in a neighborhood. Perform univariate analysis on this dataset using Python and calculate the mean, median, mode, and standard deviation of the house prices. Additionally, plot a box plot to visualize the distribution of prices. Write a Python program to implement this analysis. house_prices = [300000, 350000, 320000, 280000, 400000, 380000, 330000,310000, 290000, 270000, 350000, 380000, 370000]	2
6	A dataset containing information about the heights of students in a college. Perform univariate analysis on this dataset using Python and calculate the mean, median, mode, and standard deviation of the heights. Also, create a histogram to visualize the distribution of heights. Write a Python program to implement this analysis. heights = [165, 170, 168, 172, 175, 169, 180, 160, 165, 172, 168, 176, 170, 173, 165]	2
7	Write a Python program to perform linear regression on a diabetes dataset and plot the regression line.	2
8	Write a python program to Create a binary classification model using Logistic Regression learn to predict whether a student passes or fails based on the number of hours studied and evaluate it using accuracy Hours_Studied= 1,2,3,4,5,6 Pass_or_Fail=0,1,1,0,1,0	2
9	Extend Logistic Regression to handle a multiclass classification problem using the Iris dataset.	2
10	Perform univariate analysis on a dataset containing information about the performance of students in a school. The dataset includes variables such as student ID, test scores in different subjects, attendance, and socio-economic background. Use Python to analyze the distribution of test scores in each subject separately. Calculate the mean, median, and standard deviation for each subject and visualize the distribution using box plots.	2
11	Apply K-Means clustering on the Iris dataset (without using labels) and visualize the clusters using the first two features.	2
12	Create a distance matrix using Manhattan and Euclidean distances. Visualize it using a heatmap (e.g., with seaborn).	2
13	Use Agglomerative Clustering on the Iris dataset.	2
14	Revision	2
15	Laboratory Test	2

1) Generate 1000 samples from a uniform continuous distribution between 0 and 1. Plot the PDF (Probability Density Function) using a histogram and overlay the theoretical PDF using python program

```
import numpy as np
import matplotlib.pyplot as plt
# Generate 1000 samples from a uniform distribution between 0 and 1
samples = np.random.uniform(0, 1, 1000)
# Plot histogram (PDF approximation)
plt.hist(samples, bins=30, density=True, alpha=0.6, color='skyblue', edgecolor='black', label="Empirical PDF")
# Theoretical PDF for uniform(0,1): f(x) = 1 for 0 <= x <= 1
x = np.linspace(0, 1, 100)
y = np.ones_like(x)
plt.plot(x, y, 'r-', linewidth=2, label="Theoretical PDF")
# Labels and legend
plt.title("PDF of Uniform(0,1) Distribution")
plt.xlabel("Value")
plt.ylabel("Density")
plt.legend()
plt.grid(True)
plt.show()
```

O/P

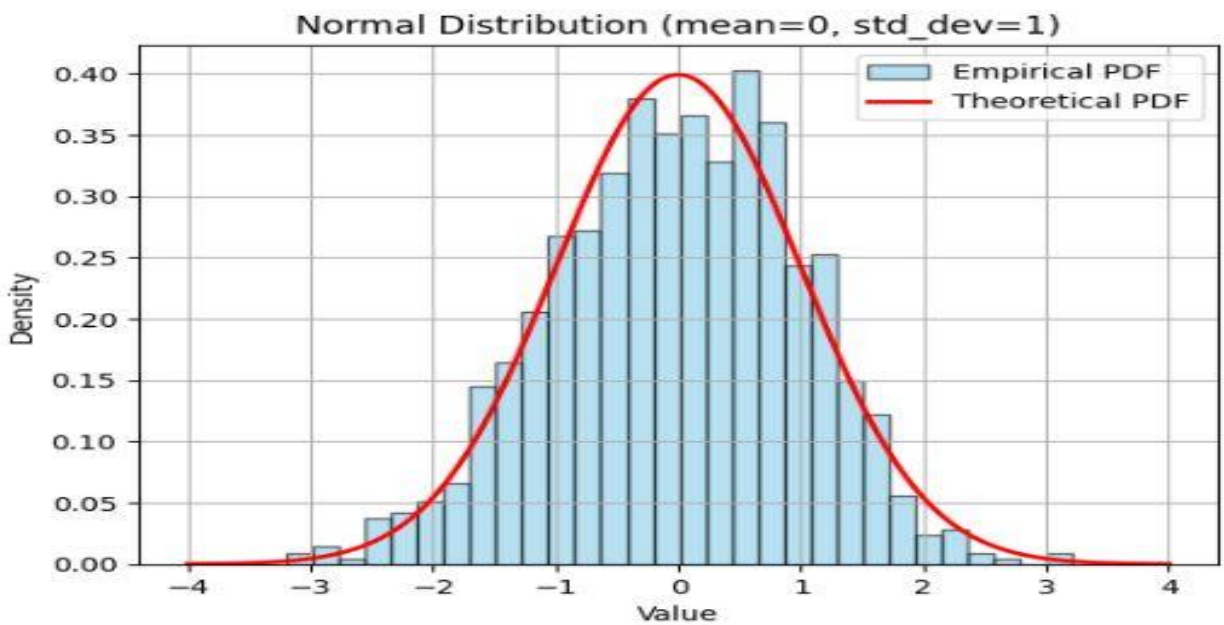


2) **Generate 1000 samples from a normal distribution with a given mean and standard deviation using python program**

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
# Parameters for Normal distribution
mean = 0
std_dev = 1
# Generate 1000 samples
samples = np.random.normal(mean, std_dev, 1000)
```

```
# Plot histogram (PDF approximation)
plt.hist(samples, bins=30, density=True, alpha=0.6, color='skyblue', edgecolor='black', label="Empirical PDF")
# Theoretical PDF
x = np.linspace(mean - 4*std_dev, mean + 4*std_dev, 200)
y = norm.pdf(x, mean, std_dev)
plt.plot(x, y, 'r-', linewidth=2, label="Theoretical PDF")
# Labels and legend
plt.title(f"Normal Distribution (mean={mean}, std_dev={std_dev})")
plt.xlabel("Value")
plt.ylabel("Density")
plt.legend()
plt.grid(True)
plt.show()
```

O/P



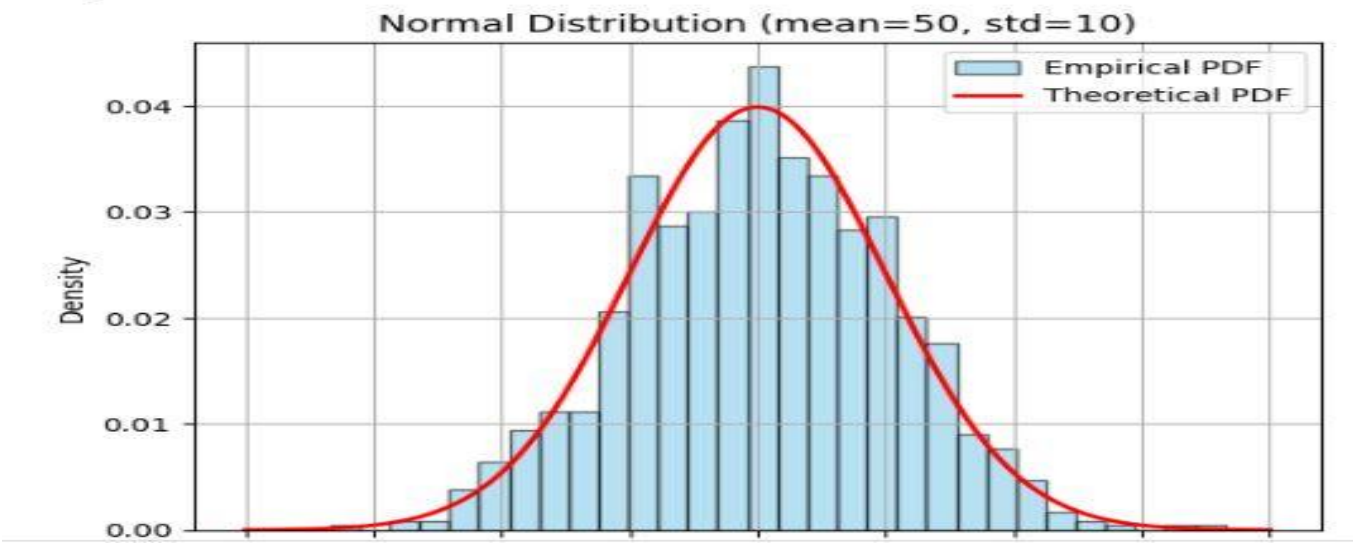
3) **Write a Python program to generate 1000 samples from a Normal distribution (mean = 50, std = 10), plot the histogram with the theoretical normal curve, and display the sample mean and standard deviation.**

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
# Parameters for Normal distribution
mean = 50
std_dev = 10
# Generate 1000 samples
samples = np.random.normal(mean, std_dev, 1000)
# Calculate sample mean and standard deviation
sample_mean = np.mean(samples)
sample_std = np.std(samples)
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Sample Standard Deviation: {sample_std:.2f}")
```

```
# Plot histogram (PDF approximation)
plt.hist(samples, bins=30, density=True, alpha=0.6, color='skyblue', edgecolor='black', label="Empirical PDF")
# Theoretical PDF
x = np.linspace(mean - 4*std_dev, mean + 4*std_dev, 200)
y = norm.pdf(x, mean, std_dev)
plt.plot(x, y, 'r-', linewidth=2, label="Theoretical PDF")
# Labels and legend
plt.title("Normal Distribution (mean=50, std=10)")
plt.xlabel("Value")
plt.ylabel("Density")
plt.legend()
plt.grid(True)
plt.show()
```

O/P

```
Sample Mean: 50.17
Sample Standard Deviation: 10.12
```

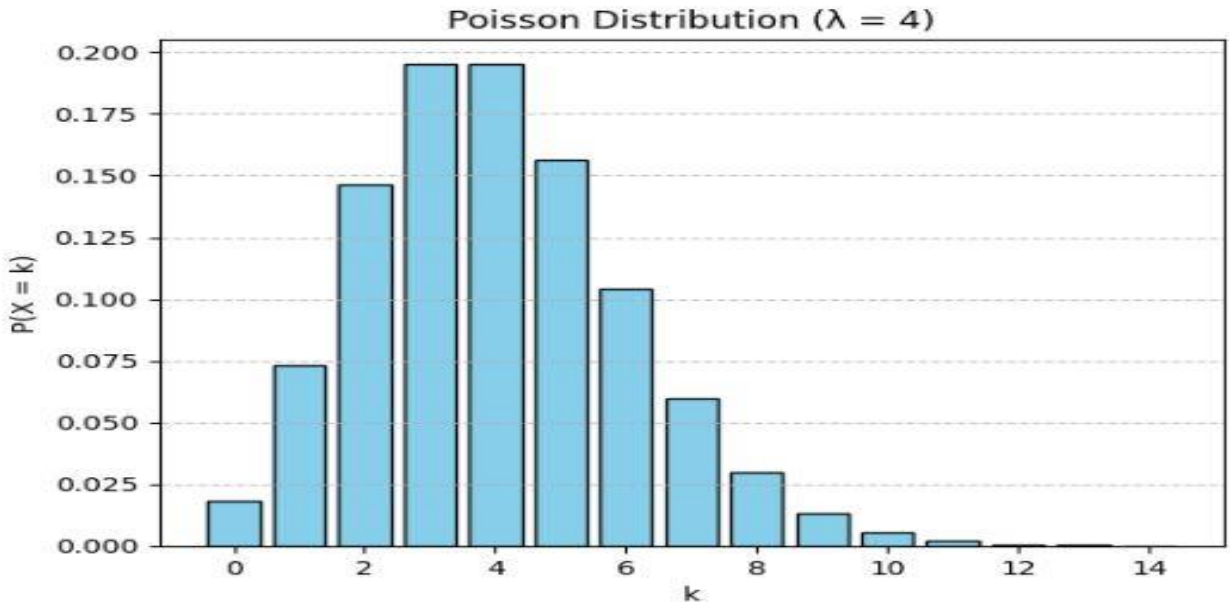


4) **Write a Python program to generate and plot the Poisson distribution for λ = 4 using scipy.stats.poisson, and visualize the probability mass function (PMF) with a bar chart.**

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson
# Parameter for Poisson distribution
lam = 4
# Define range of x values (0 to 3*lambda for good coverage)
x = np.arange(0, 15)
# PMF values
pmf_values = poisson.pmf(x, lam)
# Plot PMF as bar chart
plt.bar(x, pmf_values, color='skyblue', edgecolor='black')
# Labels and title
plt.title("Poisson Distribution (λ = 4)")
plt.xlabel("k")
```

```
plt.ylabel("P(X = k)")
# Display grid
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

O/P



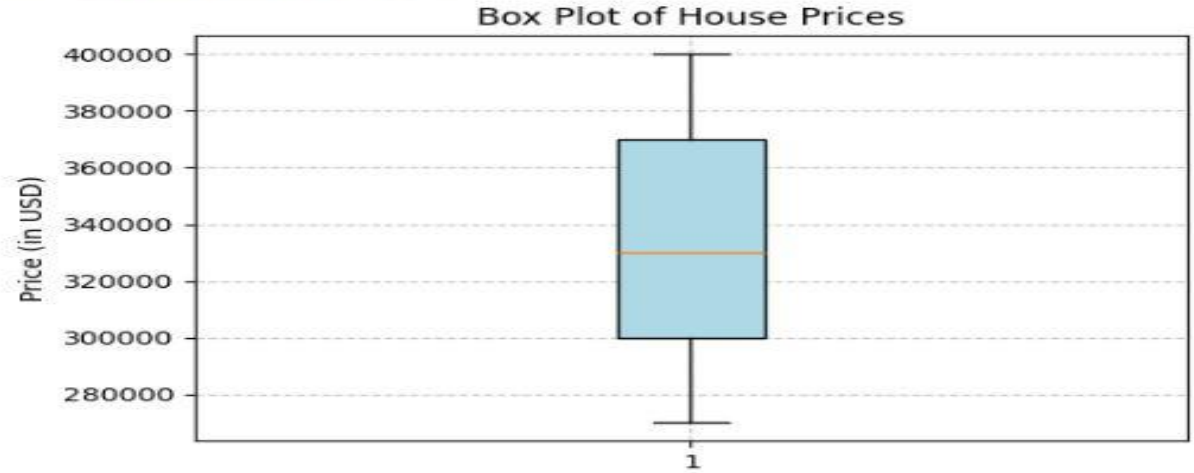
5) **A dataset containing the prices of houses in a neighborhood. Perform univariate analysis on this dataset using Python and calculate the mean, median, mode, and standard deviation of the house prices. Additionally, plot a box plot to visualize the distribution of prices. Write a Python program to implement this analysis. house_prices = [300000, 350000, 320000, 280000, 400000, 380000, 330000,310000, 290000, 270000, 350000, 380000, 370000]**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statistics import mode, StatisticsError
# Dataset
house_prices = [300000, 350000, 320000, 280000, 400000, 380000,
            330000, 310000, 290000, 270000, 350000, 380000, 370000]
# Convert to pandas Series for convenience
prices = pd.Series(house_prices)
# Calculate statistics
mean_price = np.mean(prices)
median_price = np.median(prices)
try:
   mode_price = mode(prices)
except StatisticsError:
   mode_price = "No unique mode"
std_dev = np.std(prices, ddof=1)  # sample standard deviation
# Display results
print("Univariate Analysis of House Prices")
print("-----------------------------------")
```

```
print(f"Mean: {mean_price:.2f}")
print(f"Median: {median_price:.2f}")
print(f"Mode: {mode_price}")
print(f"Standard Deviation: {std_dev:.2f}")
# Plot box plot
plt.figure(figsize=(6, 4))
plt.boxplot(prices, vert=True, patch_artist=True, boxprops=dict(facecolor='lightblue'))
plt.title("Box Plot of House Prices")
plt.ylabel("Price (in USD)")
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

O/P

```
Univariate Analysis of House Prices
-----------------------------------
Mean: 333076.92
Median: 330000.00
Mode: 350000
Standard Deviation: 42107.92
```



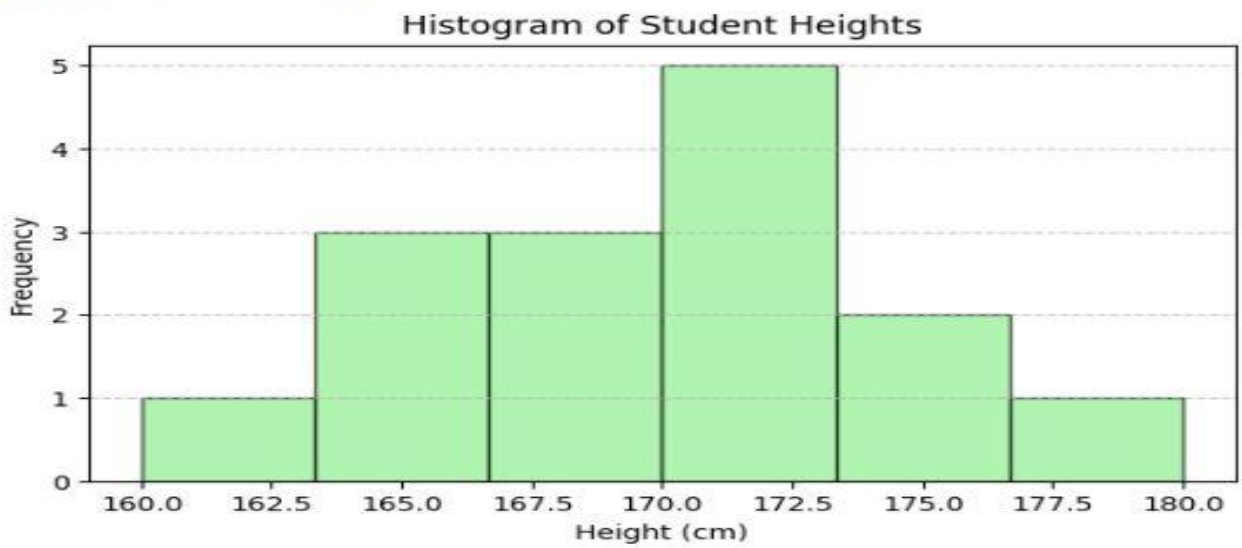
6) A dataset containing information about the heights of students in a college. Perform univariate analysis on this dataset using Python and calculate the mean, median, mode, and standard deviation of the heights. Also, create a histogram to visualize the distribution of heights. Write a Python program to implement this analysis. heights = [165, 170, 168, 172, 175, 169, 180, 160, 165, 172, 168, 176, 170, 173, 165]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statistics import mode, StatisticsError
# Dataset
heights = [165, 170, 168, 172, 175, 169, 180, 160, 165, 172, 168, 176, 170, 173, 165]
# Convert to pandas Series for convenience
height_series = pd.Series(heights)
# Calculate statistics
mean_height = np.mean(height_series)
median_height = np.median(height_series)
try:
    mode_height = mode(height_series)
```

```
except StatisticsError:
    mode_height = "No unique mode"
std_dev_height = np.std(height_series, ddof=1)  # sample standard deviation
# Display results
print("Univariate Analysis of Student Heights")
print("--------------------------------------")
print(f"Mean: {mean_height:.2f} cm")
print(f"Median: {median_height:.2f} cm")
print(f"Mode: {mode_height} cm")
print(f"Standard Deviation: {std_dev_height:.2f} cm")
# Plot histogram
plt.figure(figsize=(7, 4))
plt.hist(height_series, bins=6, color='lightgreen', edgecolor='black', alpha=0.7)
plt.title("Histogram of Student Heights")
plt.xlabel("Height (cm)")
plt.ylabel("Frequency")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

O/P

```
Univariate Analysis of Student Heights
--------------------------------------
Mean: 169.87 cm
Median: 170.00 cm
Mode: 165 cm
Standard Deviation: 5.08 cm
```

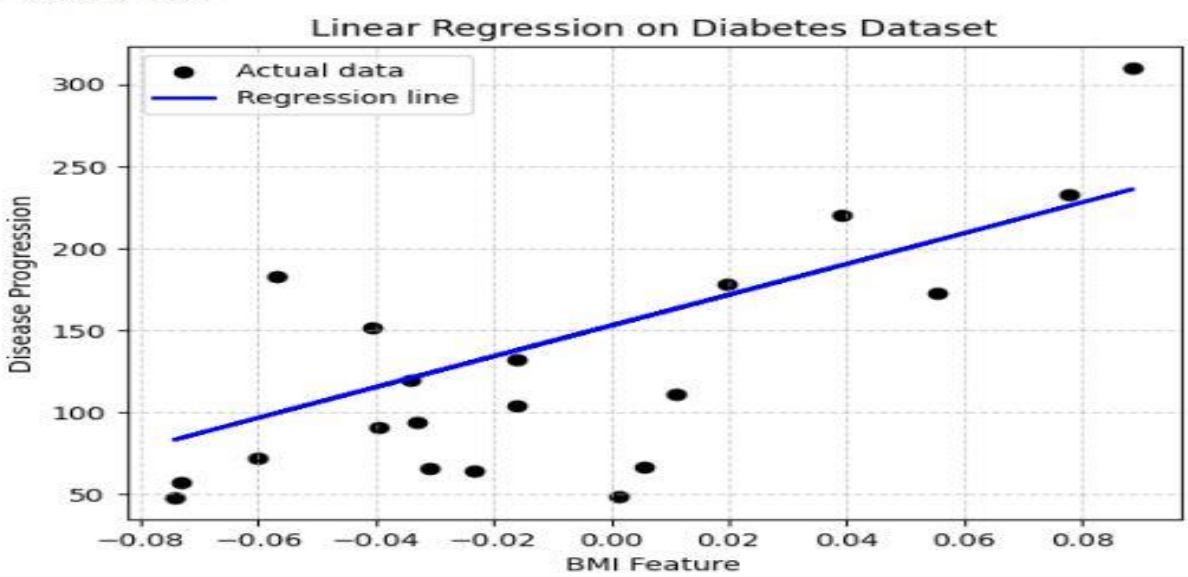


7) Write a Python program to perform linear regression on a diabetes dataset and plot the regression line.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score
# Load the diabetes dataset
diabetes = datasets.load_diabetes()
# Use only one feature (for 2D visualization)
X = diabetes.data[:, np.newaxis, 2]  # Using BMI feature
y = diabetes.target
```

```
# Split the data into training/testing sets
X_train = X[:-20]
X_test = X[-20:]
y_train = y[:-20]
y_test = y[-20:]
# Create linear regression model
model = linear_model.LinearRegression()
# Train the model
model.fit(X_train, y_train)
# Make predictions
y_pred = model.predict(X_test)
# Display results
print("Linear Regression on Diabetes Dataset")
print("-----------------------------------")
print(f"Coefficients: {model.coef_}")
print(f"Intercept: {model.intercept_:.2f}")
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}")
print(f"R² Score: {r2_score(y_test, y_pred):.2f}")
# Plot outputs
plt.scatter(X_test, y_test, color='black', label='Actual data')
plt.plot(X_test, y_pred, color='blue', linewidth=2, label='Regression line')
plt.title("Linear Regression on Diabetes Dataset")
plt.xlabel("BMI Feature")
plt.ylabel("Disease Progression")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

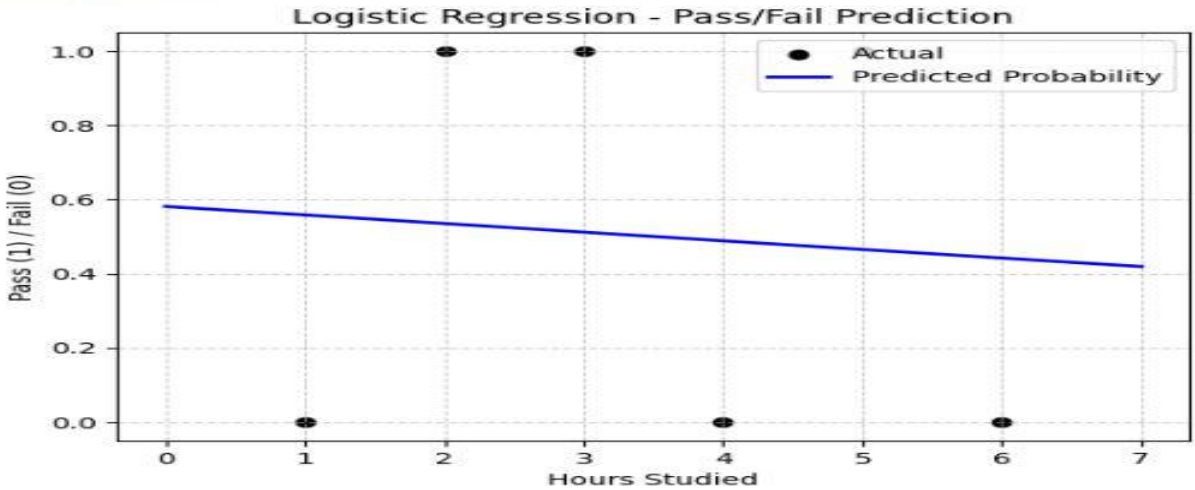
```
Linear Regression on Diabetes Dataset
-----------------------------------
Coefficients: [938.23786125]
Intercept: 152.92
Mean Squared Error: 2548.07
R² Score: 0.47
```



8) Write a python program to Create a binary classification model using Logistic Regression learn to predict whether a student passes or fails based on the number of hours studied and evaluate it using accuracy Hours_Studied= 1,2,3,4,5,6 Pass_or_Fail=0,1,1,0,1,0

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
# Dataset
Hours_Studied = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
Pass_or_Fail = np.array([0, 1, 1, 0, 1, 0])  # 0 = Fail, 1 = Pass
# Create Logistic Regression model
model = LogisticRegression()
# Train the model
model.fit(Hours_Studied, Pass_or_Fail)
# Predict on training data
y_pred = model.predict(Hours_Studied)
# Evaluate accuracy
accuracy = accuracy_score(Pass_or_Fail, y_pred)

print("Logistic Regression Model for Student Pass/Fail Prediction")
print("------------------------------------------------------------")
print(f"Coefficients: {model.coef_}")
print(f"Intercept: {model.intercept_}")
print(f"Accuracy: {accuracy:.2f}")
# Plot data points
plt.scatter(Hours_Studied, Pass_or_Fail, color='black', label='Actual')
plt.xlabel("Hours Studied")
plt.ylabel("Pass (1) / Fail (0)")
plt.title("Logistic Regression - Pass/Fail Prediction")

# Plot logistic regression curve
X_test = np.linspace(0, 7, 100).reshape(-1, 1)
y_prob = model.predict_proba(X_test)[:, 1]
plt.plot(X_test, y_prob, color='blue', linewidth=2, label='Predicted Probability')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
Logistic Regression Model for Student Pass/Fail Prediction
------------------------------------------------------------
Coefficients: [[-0.09330877]]
Intercept: [0.326593]
Accuracy: 0.67
```

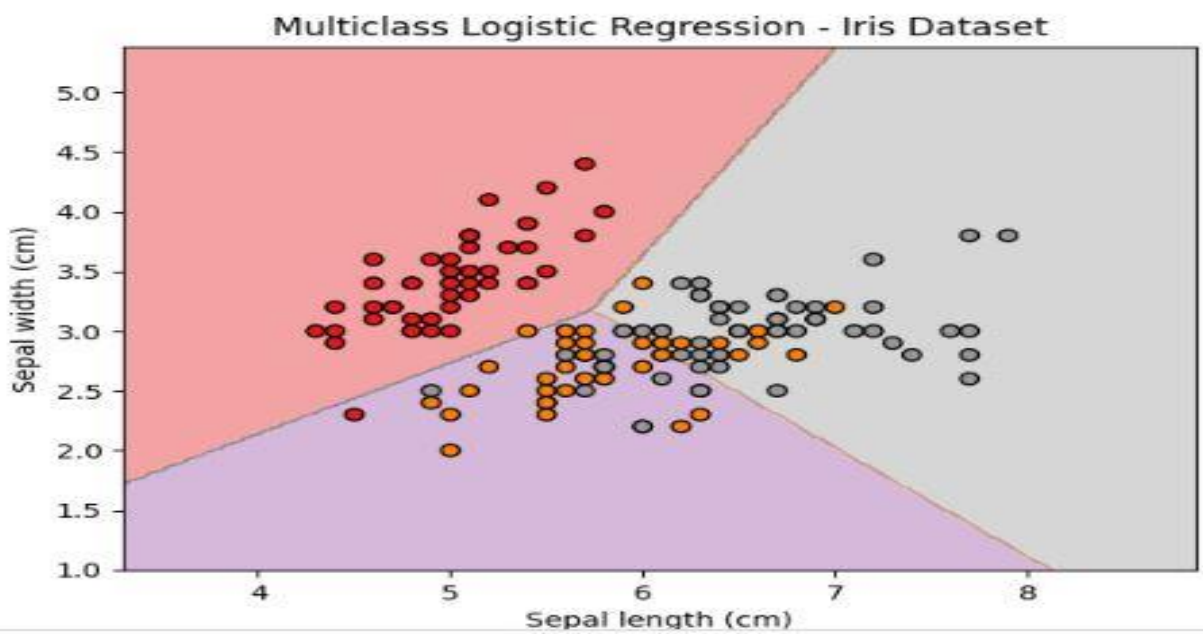


9) Extend Logistic Regression to handle a multiclass classification problem using the Iris dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Load Iris dataset
iris = datasets.load_iris()
X = iris.data[:, :2]   # Use first two features (sepal length, sepal width) for visualization
y = iris.target

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create Logistic Regression model (multiclass)
model = LogisticRegression(multi_class='ovr', solver='lbfgs', max_iter=200)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)
print("Multiclass Logistic Regression on Iris Dataset")
print("---------------------------------------------")
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Visualization of decision boundaries
```

```
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

# Predict classes for each point in the grid
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot decision boundary
plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.Set1)
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k', cmap=plt.cm.Set1)
plt.xlabel('Sepal length (cm)')
plt.ylabel('Sepal width (cm)')
plt.title('Multiclass Logistic Regression - Iris Dataset')
plt.show()
```

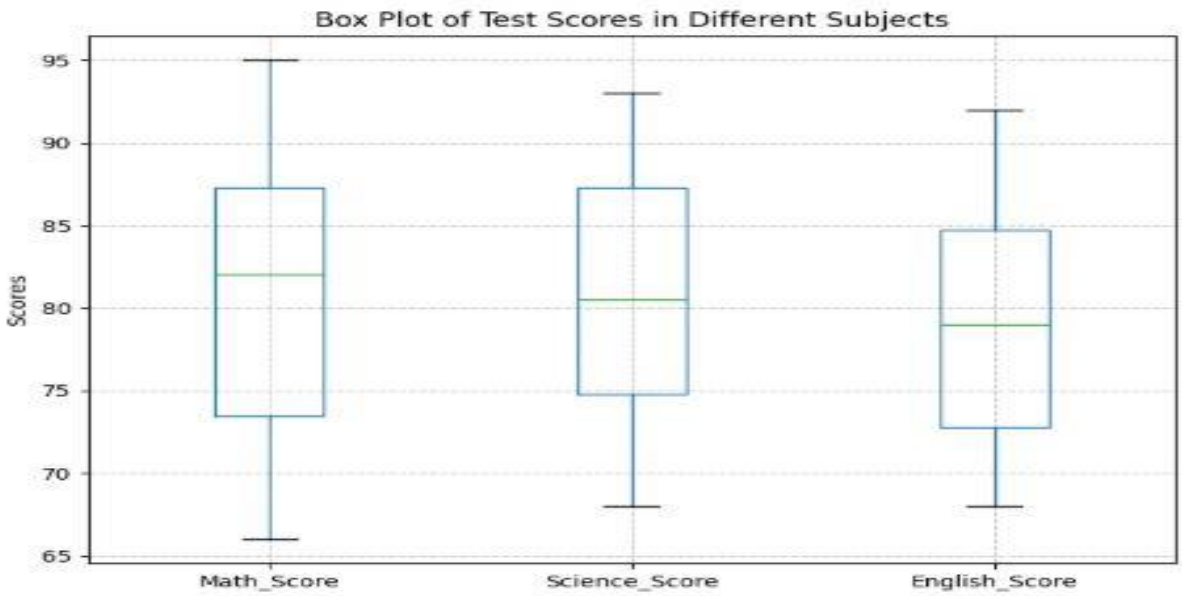
```
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       0.78      0.54      0.64        13
   virginica       0.65      0.85      0.73        13

    accuracy                           0.82        45
   macro avg       0.81      0.79      0.79        45
weighted avg       0.83      0.82      0.82        45
```



10) Perform univariate analysis on a dataset containing information about the performance of students in a school. The dataset includes variables such as student ID, test scores in different subjects, attendance, and socio-economic background. Use Python to analyze the distribution of test scores in each subject separately. Calculate the mean, median, and standard deviation for each subject and visualize the distribution using box plots.

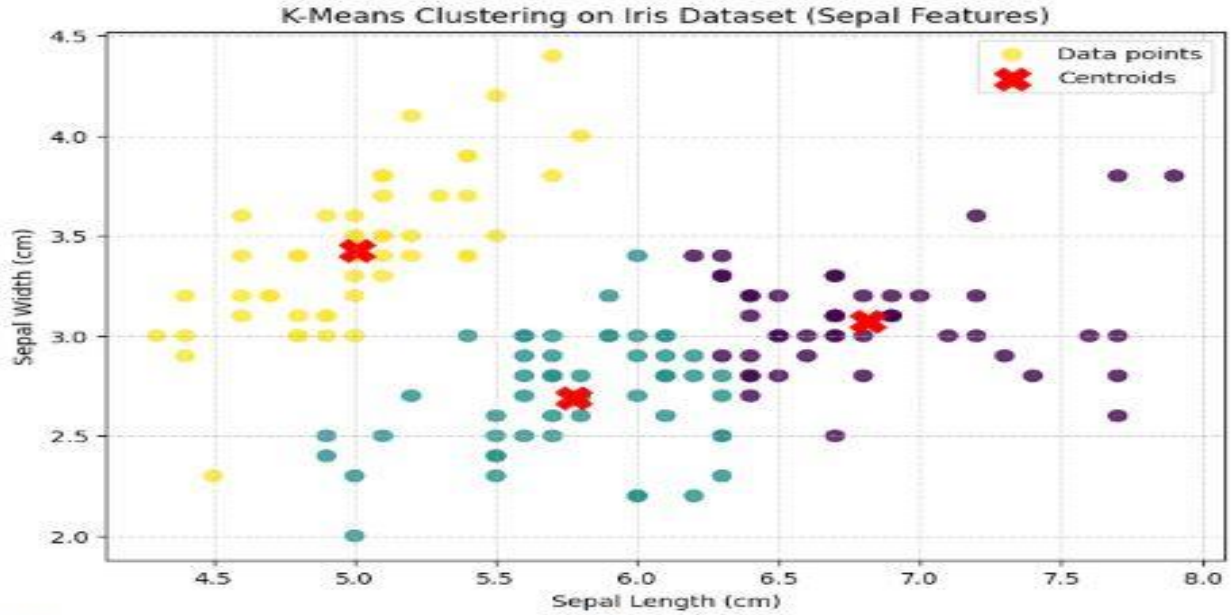
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# Create a sample dataset
data = {
    'Student_ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Math_Score': [78, 85, 90, 66, 72, 88, 95, 80, 70, 84],
    'Science_Score': [82, 79, 91, 68, 74, 85, 93, 77, 73, 88],
    'English_Score': [75, 80, 85, 70, 68, 90, 92, 78, 72, 84],
    'Attendance': [90, 85, 88, 80, 92, 95, 98, 85, 87, 90],
    'Socio_Economic_Status':  ['High',  'Medium',  'Low',  'Medium',  'Low',  'High',  'High',  'Medium',
'Low', 'High']
}

# Create DataFrame
df = pd.DataFrame(data)

# Select test score columns
subjects = ['Math_Score', 'Science_Score', 'English_Score']

print("Univariate Analysis of Student Test Scores")
print("----------------------------------------")

# Calculate and display statistics for each subject
for subject in subjects:
    mean_val = df[subject].mean()
    median_val = df[subject].median()
    std_dev = df[subject].std()

    print(f"{subject}:")
    print(f"  Mean: {mean_val:.2f}")
    print(f"  Median: {median_val:.2f}")
    print(f"  Standard Deviation: {std_dev:.2f}\n")

# Visualize distributions using box plots
plt.figure(figsize=(8, 6))
df[subjects].boxplot()
plt.title("Box Plot of Test Scores in Different Subjects")
plt.ylabel("Scores")
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



11) Apply K-Means clustering on the Iris dataset (without using labels) and visualize the clusters using the first two features.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # Use only first two features (sepal length & sepal width)

# Apply K-Means clustering (assuming 3 clusters for 3 flower types)
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

# Get cluster labels and centroids
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Plot the clustered data
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50, alpha=0.8, label='Data points')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=200, label='Centroids')
plt.title("K-Means Clustering on Iris Dataset (Sepal Features)")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
# Display centroid coordinates
print("Cluster Centroids (Sepal Length, Sepal Width):")
print(centroids)
```

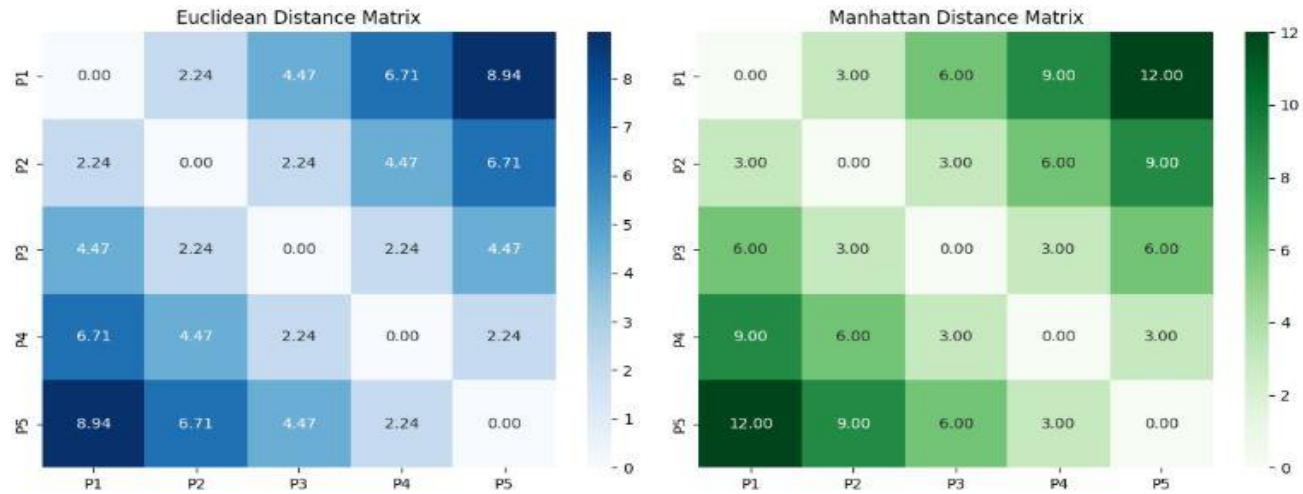



```
Cluster Centroids (Sepal Length, Sepal Width):
[[6.81276596 3.07446809]
 [5.77358491 2.69245283]
 [5.006      3.428     ]]
```

12) Create a distance matrix using Manhattan and Euclidean distances. Visualize it using a heatmap (e.g., with seaborn).

```
# Import required libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.spatial.distance import cdist

# Create a simple dataset (e.g., 5 points with 2 features each)
data = np.array([
    [1, 2],
    [2, 4],
    [3, 6],
    [4, 8],
    [5, 10]
])

# Convert to DataFrame for better readability
df = pd.DataFrame(data, columns=["Feature_1", "Feature_2"])

# Compute distance matrices
euclidean_dist = cdist(df, df, metric='euclidean')
manhattan_dist = cdist(df, df, metric='cityblock')

# Create DataFrames for labeled distance matrices
euclidean_df = pd.DataFrame(euclidean_dist,
                            columns=[f"P{i}" for i in range(1, 6)],
                            index=[f"P{i}" for i in range(1, 6)])
```

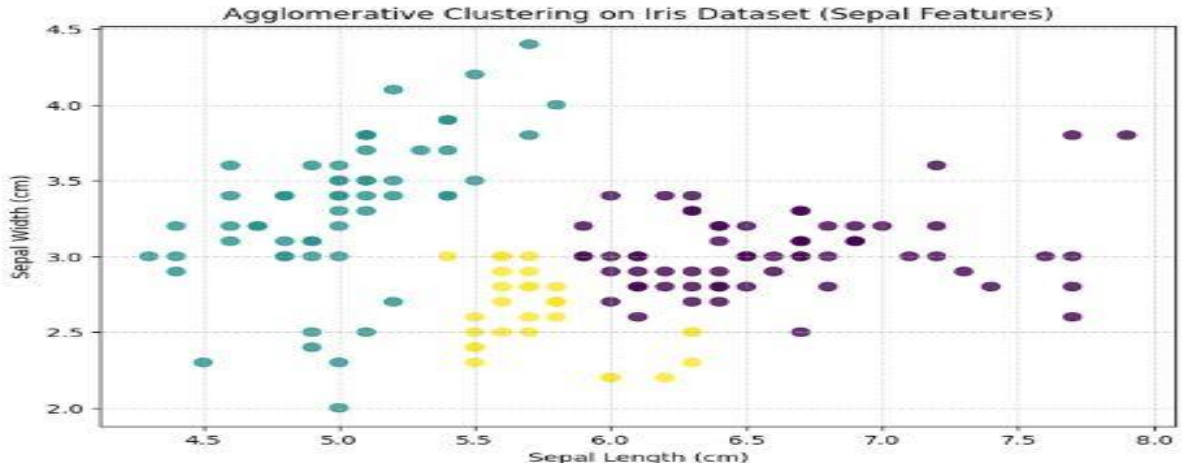
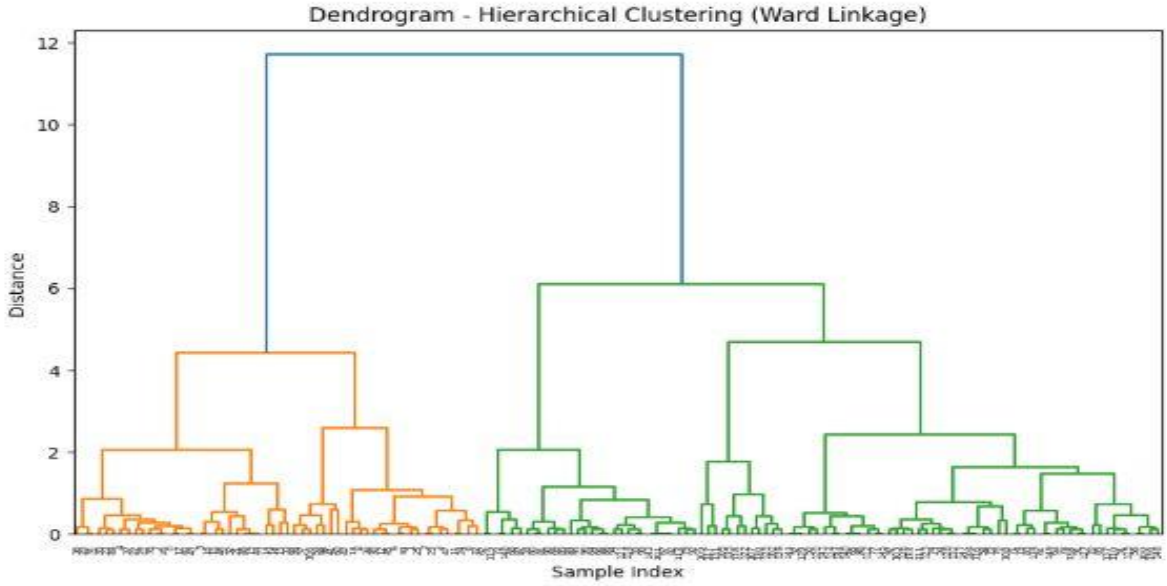
```
manhattan_df = pd.DataFrame(manhattan_dist, columns=[f"P{i}" for i in range(1, 6)], index=[f"P{i}" for i in range(1, 6)])

# Plot heatmaps for both distance matrices
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.heatmap(euclidean_df, annot=True, cmap="Blues", fmt=".2f")
plt.title("Euclidean Distance Matrix")

plt.subplot(1, 2, 2)
sns.heatmap(manhattan_df, annot=True, cmap="Greens", fmt=".2f")
plt.title("Manhattan Distance Matrix")

plt.tight_layout()
plt.show()
```



13) Use Agglomerative Clustering on the Iris dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # Use first two features for visualization (sepal length, sepal width)

# Perform Agglomerative Clustering
agg_cluster = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = agg_cluster.fit_predict(X)

# Visualize clusters
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50, alpha=0.8)
plt.title("Agglomerative Clustering on Iris Dataset (Sepal Features)")
```

```
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

 # Create a dendrogram to visualize the hierarchical clustering

```
linked = linkage(X, method='ward')

plt.figure(figsize=(10, 6))
dendrogram(linked,
        orientation='top',
        distance_sort='ascending',
        show_leaf_counts=True)
plt.title("Dendrogram - Hierarchical Clustering (Ward Linkage)")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.show()
```